

CrimeLink

Submitted by:-- Nihira Sinha,2212427

Sanya Bharadwaj,2212451

Shruti Kumari,2212464

Tushti Savarn,2212478

(GROUP-8)

For the award of the degree of
BCA (Bachelor of Computer Applications)

Under the supervision of

Supervisor

(Mr.Manoj Kumar Shukla)

(Ms.Aishvarya Garg)



Department of Mathematics and Computing

Banasthali Vidyapith

Banasthali - 304022

Session: 2024-2025

CERTIFICATE

Certified that Nihira Sinha, Sanay Bharadwaj, Shruti Kumari, Tushti Savarn has carried out the project work titled "CrimeLink" from 2/1/25 to 30/4/25 for the award of the BCA (Bachelor of Computer Applications) from Banasthali Vidhyapith under my supervision.

The report embodies results of original work and studies carried out by the student herself and the contents of the thesis do not form the basis for the award of any other degree to the candidate or to anybody else.

(Name of the Supervisor)

Mr. Manoj Kumar Shukla

Ms. Aishvarya Garg

Banasthali Vidhyapith

30/4/25

ACKNOWLEDGEMENT

We sincerely express our heartfelt gratitude to all those who have supported and contributed to the successful completion of our project, "CrimeLink", a crime reporting website.

First and foremost, we express our sincere appreciation to Mr. Manoj Kumar Shukla and Ms. Aishvarya Garg for their invaluable guidance, insightful feedback, and unwavering support throughout the development process. Their expertise and mentorship played a crucial role in refining our approach and enhancing the quality of our work.

We also extend our deepest gratitude to our team members for their dedication, technical proficiency, and collaborative spirit. Each member's commitment, innovative thinking, and problem-solving skills were instrumental in transforming our vision into a functional and efficient platform.

Furthermore, we acknowledge our institution for providing us with the resources and a conducive learning environment, enabling us to apply our theoretical knowledge in a practical setting. We also appreciate the encouragement and motivation from our family and friends, whose support kept us driven throughout this journey.

Lastly, we recognize the contributions of various online resources, open-source technologies, and industry best practices that have enriched our understanding and execution of this project. This experience has been an invaluable learning opportunity, equipping us with essential skills and insights for future endeavours.

ABSTRACT

CrimeLink, Online Crime Reporting Platform is designed as a centralized, user-friendly, and secure web-based system to streamline the process of reporting, managing, and tracking crimes. It bridges the gap between citizens, law enforcement agencies, and administrative entities, fostering efficient communication and resolution of issues. This platform modernizes traditional, often cumbersome methods of crime reporting—such as physical visits to police stations or phone calls—by providing a digital solution that is accessible, transparent, and convenient. The system reduces paperwork, minimizes response times, and enables citizens to report crimes, upload evidence, and track their cases in real-time. The system forms a part of broader efforts to digitize and modernize law enforcement operations.

It integrates seamlessly with existing police databases and government communication networks, while also supporting advanced features like crime categorization and analysis. The platform is accessible via both desktop and mobile devices, ensuring inclusivity and ease of use for a diverse range of users. With features like real-time status tracking, AI-powered chatbot assistance, and an emergency SOS function, this system aims to enhance public safety, improve user satisfaction, and optimize the efficiency of law enforcement processes.

TABLE OF CONTENTS

- Cover Page & Title Page
- Certificate from Mentor
- Acknowledgement
- Abstract
- 1. Introduction
 - 1.1 Purpose
 - 1.2 How it works
- 2. Requirement Analysis(SRS)
 - 2.1 Requirement specification
 - 2.1.1 Functional Requirements
 - 2.1.2 Non Functional Requirements
 - 2.2 Hardware and Software Requirements
 - 2.3 Feasibility Study
 - 2.4 Product Functions
 - 2.5 Use-case Diagrams
- 3. System Design (SDS)
 - 3.1 High-level Design
 - 3.2 ER Diagram
 - 3.3 Database Design
 - 3.4 Activity Diagrams
 - 3.5 Sequence Diagrams
- 4. Coding
- 5. Testing
- 6. User Interfaces
- 7. Appendices
- 8. References

1. Introduction

The Online Crime Reporting System (CrimeLink) is a web-based application designed to facilitate the reporting and management of criminal activities. It provides a secure, efficient platform for citizens to report crimes, upload evidence, and track their reports in real time, while enabling law enforcement agencies to manage and respond effectively.

1.1 Purpose

The purpose of the CrimeLink system is:

- To simplify and speed up the crime reporting process.
- To allow citizens to report crimes online easily, upload evidence (photos, videos, documents), and track the status of their complaints.
- To modernize police departments by transitioning from manual record-keeping to a computerized, online system.
- To enhance transparency, reduce paperwork, and improve police responsiveness.

1.2 How It Works

Although the original PDF specifically mentioned "Purpose" and "Scope" separately and did not title anything directly as "How it Works," based on the information in the document, here's a concise write-up for How it Works:

The CrimeLink platform operates as follows:

- Citizens register or log in securely using a web interface.
- They report crimes by filling an online form, upload evidence (images, videos, documents), and submit the report.
- Each report is assigned a unique case ID.
- Police officers can review reports, assign cases to teams, update statuses (Under Review, In Progress, Closed), and communicate with citizens if needed.
- Citizens can track the real-time status of their submitted cases.

- An emergency SOS feature allows users to quickly alert nearby police stations with live location information.
- An AI-powered chatbot is integrated to help users with FAQs and assist them through the reporting process.
- Data encryption and security protocols protect user information and case data.
- Administrators oversee the system operations, user management, and generate analytical reports to monitor crime trends.

2. Requirement Analysis (SRS)

The Software Requirement Specification (SRS) outlines all the functional and non-functional needs of the CrimeLink system. It ensures that the system meets user expectations and technical standards.

2.1 Requirement Specification

The CrimeLink system needs to fulfill both functional and non-functional requirements to operate successfully.

2.1.1 Functional Requirements

These are the specific behaviors and functions the system must perform:

- User Registration and Authentication:
 - Secure account creation for citizens, police, and administrators.
 - Role-based access (different permissions for citizens, police officers, and admins).
- Crime Reporting:
 - Citizens can submit reports using guided forms.
 - Support for uploading evidence (photos, videos, documents).
 - Option to report crimes anonymously.
- Case Management:

- Automatic generation of unique case IDs for each report.
 - Police officers can update the status of cases (e.g., Under Review, In Progress, Closed).
- Emergency SOS Feature:
 - Allows users to send emergency alerts with real-time location sharing.
 - Immediate notifications to the nearest police station or emergency responders.
- Case Tracking:
 - Citizens can monitor the progress of their crime reports online.
- Admin Panel:
 - Admins can manage user database.
- Chatbot Assistance:
 - AI-powered chatbot helps users report crimes, navigate the system, and answers FAQs.

2.1.2 Non-Functional Requirements

These define how the system performs and the qualities it must possess:

- Performance:
 - Real-time database updates for case tracking.
- Security:
 - All data transmission must be encrypted using HTTPS.
 - Passwords stored using hashing algorithms.

- Scalability:
 - The system should easily support a growing number of users and reports without performance degradation.
 - Flexible architecture to integrate additional features and third-party services.
- Usability:
 - Simple, intuitive user interfaces for all types of users (citizens, police officers, administrators).
 - Fully responsive design — accessible via desktop, tablets, and mobile phones.
 - Easy-to-navigate system with guided reporting processes.

2.2 Hardware and Software Requirements

This section outlines the necessary hardware and software components for the proper development, deployment, and use of the CrimeLink system.

2.2.1 Hardware Requirements

Client-Side Requirements

Supported Devices: Desktop computers, laptops, smartphones, and tablets.

Minimum Specifications:

Processor: Dual-core (Intel i3 or equivalent) or higher.

RAM: Minimum 2 GB.

Storage: At least 500 MB of free space (for browser caching).

Internet Speed: Minimum 2 Mbps for smooth user experience.

Browser Compatibility: Chrome, Firefox, Edge, Safari (latest versions).

Server-Side Requirements

Processor: Quad-core (Intel Xeon or AMD Ryzen) or higher.

RAM: Minimum 8 GB (to handle concurrent users and large data processing).

Storage: At least 100 GB SSD (for storing reports, evidence files, logs).

Bandwidth: Minimum 100 Mbps (to support multiple users simultaneously).

Database Support: MySQL

2.2.2 Software Requirements

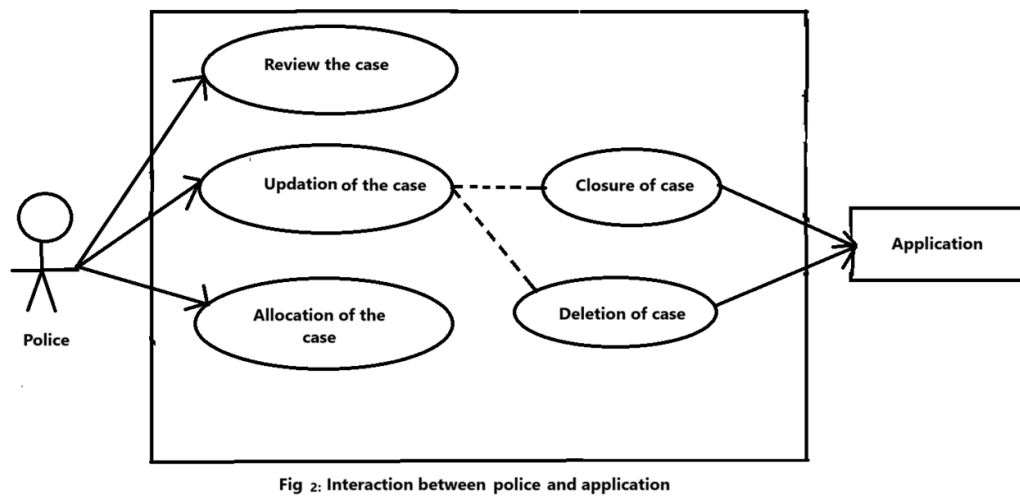
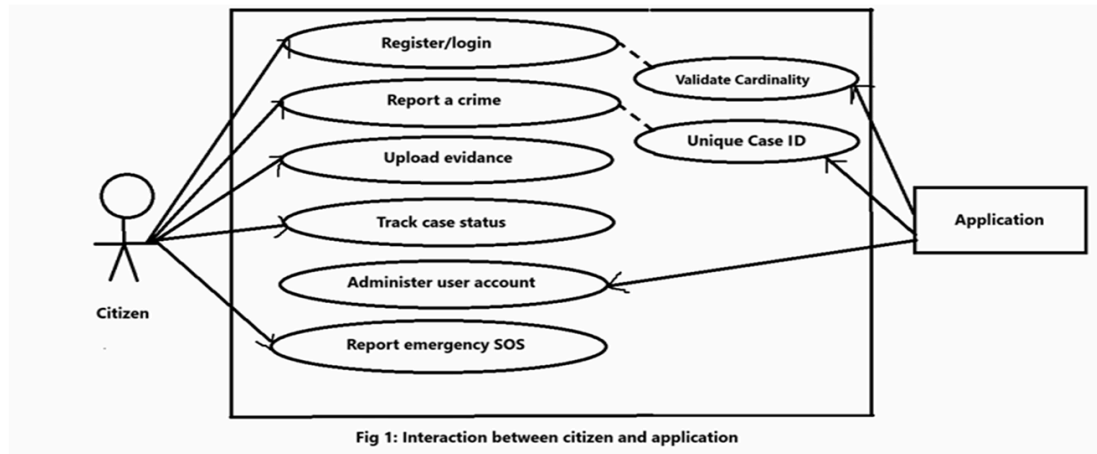
Backend

Server-Side Scripting: PHP

Frontend

User Interface Development: HTML, CSS, JavaScript
Database
Database Management System: MySQL
Chatbot Communication: Python API-driven text-based responses

2.5 Use-Case Diagrams



3. System Design (SDS)

The System Design Specification (SDS) outlines the architecture, database schema, and workflow for the CrimeLink system.

3.1 High-Level Design

CrimeLink is structured as a multi-tier web application:

- Frontend: HTML, CSS, JavaScript
- Backend: PHP
- Database: MySQL
- Communication Interface: HTTPS protocol, SMTP, SMS APIs

The system is divided into modules:

- Citizen Module: Reporting crimes, uploading evidence, tracking cases.
- Police Module: Reviewing cases, updating statuses, managing users.
- Admin Module: System monitoring, case analytics, user management.
- Emergency SOS Module: Sending instant alerts to authorities.

3.2 ER Diagram

The **Entity-Relationship Diagram** represents the main entities and their relationships:



Fig : ER Diagram of the crime reporting system

3.3 Database Design

CrimeLink's database uses **MySQL**:

- **Users Table:** Stores details of citizens, police officers, admins.
- **Reports Table:** Contains crime reports linked to users.
- **Evidence Table:** Stores uploaded files linked to specific reports.
- **Status Table:** Tracks case statuses ("Under Review", "In Progress", "Closed").

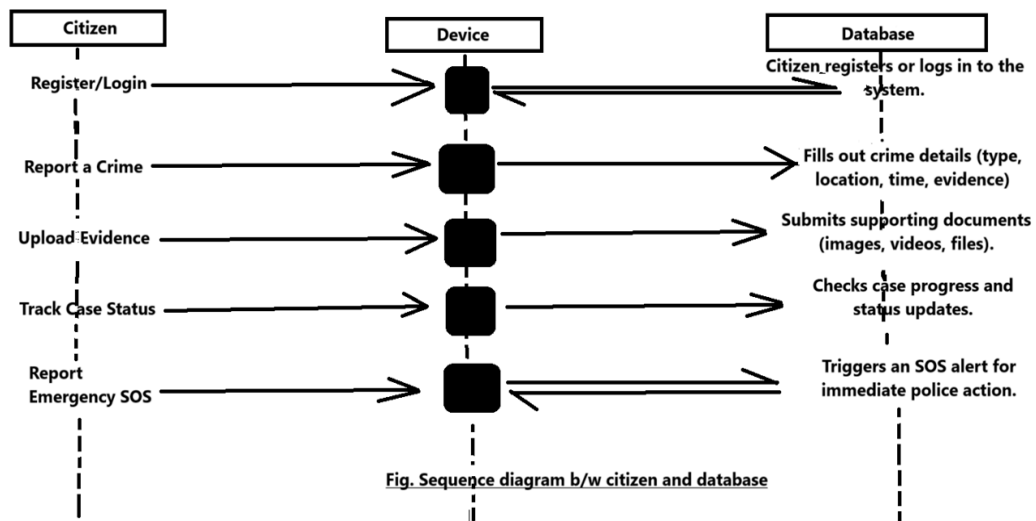
Keys:

- Primary Keys (PK) for each table (UserID, ReportID, StatusID, EvidenceID).

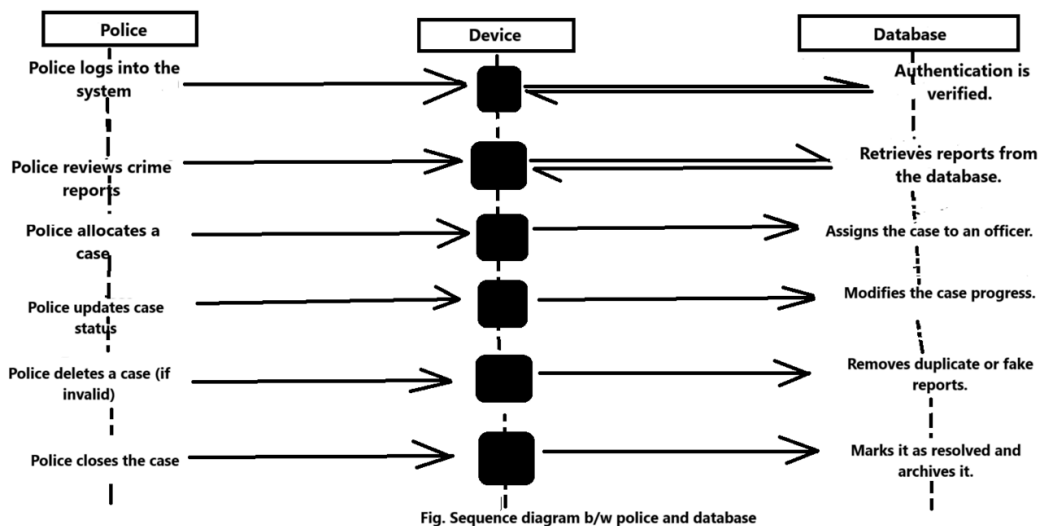
- Foreign Keys (FK) link tables together maintaining integrity.

3.4 Sequence Diagrams

Citizen Sequence:



3.5 Police Officer Sequence:



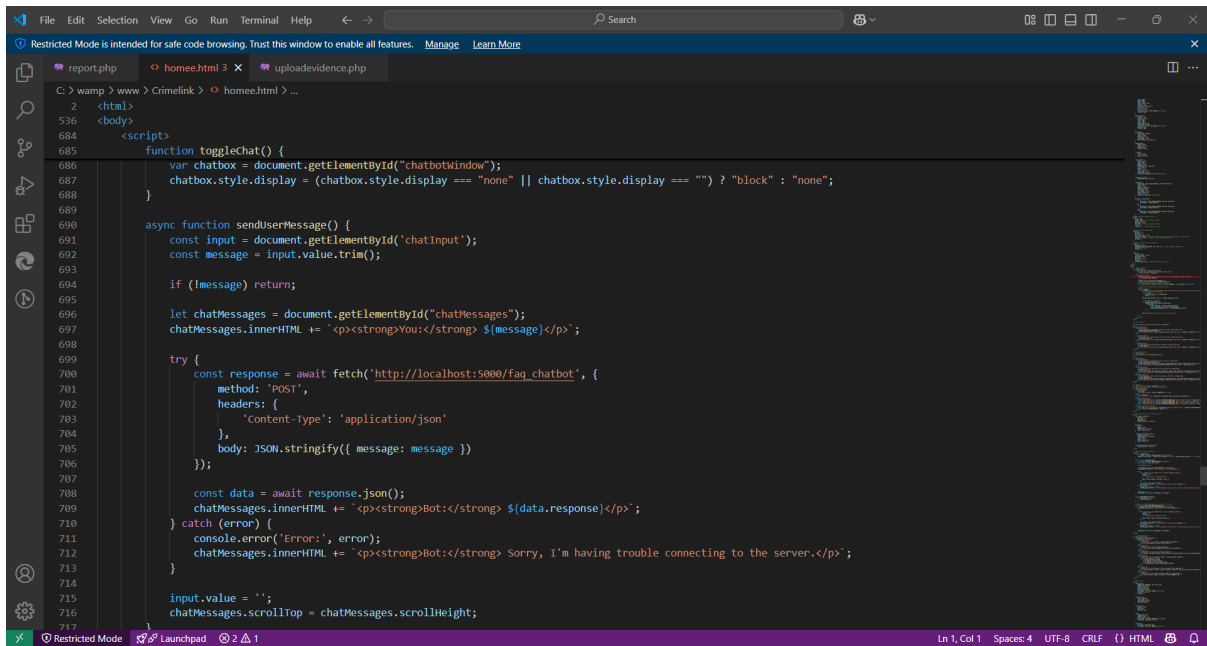
4. Coding

4.1 Code for SOS feature in citizen dashboard.

```
File Edit Selection View Go Run Terminal Help Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
home.html 2 x
C:\> wamp > www > Crimelink > home.html > ...
2 <html>
536 <body>
538 <div class="headerr">
543 <div class="nav-linkss">
547 <button id="sosButton" class="sosButton" onclick="sendSOS()"> Send SOS </button>
548
549 <!-- Add this script before the closing body tag -->
550 <script>
551 function sendSOS() {
552   if (confirm('Are you sure you want to send an SOS alert? This will notify nearby police stations.')) {
553     // Store SOS data with timestamp
554     const sosData = {
555       timestamp: new Date().toISOString(),
556       status: 'active'
557     };
558     localStorage.setItem('sosAlert', JSON.stringify(sosData));
559
560     // Get location if available
561     if (navigator.geolocation) {
562       navigator.geolocation.getCurrentPosition(
563         (position) => {
564           sosData.latitude = position.coords.latitude;
565           sosData.longitude = position.coords.longitude;
566           localStorage.setItem('sosAlert', JSON.stringify(sosData));
567         }
568       );
569     }
570     alert('SOS alert has been sent. Help is on the way!');
571   }
572 }
573 </script>
574 </div>
575 </div>
576
577 <!-- Main Content -->
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF () HTML
```

4.2 Code for Chatbot in citizen dashboard .

```
File Edit Selection View Go Run Terminal Help Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More
home.html 2 x
C:\> wamp > www > Crimelink > home.html > ...
536 <body>
627 <!-- chatbot -->
628 <div id="chatbotIcon" onclick="toggleChat()"> </div>
629 <!-- Update the chatbot window to include a text input -->
630 <div id="chatbotWindow">
631   <div id="chatHeader">
632     FAQ chatbot
633     <span class="closeBtn" onclick="toggleChat()"> </span>
634   </div>
635   <div id="chatMessages">
636     <p><strong>Bot:</strong> Please select a question below or type your own question.</p>
637   </div>
638   <div id="questionButtons">
639     <button class="question-btn" onclick="sendPredefinedMessage('How do I report a crime?')>How do I report a crime?</button>
640     <button class="question-btn" onclick="sendPredefinedMessage('How can I track my case?')>How can I track my case?</button>
641     <button class="question-btn" onclick="sendPredefinedMessage('What is SOS?')>What is SOS?</button>
642     <button class="question-btn" onclick="sendPredefinedMessage('Where to register my complaint?')>Where to register my complaint?</button>
643   </div>
644   <div class="chat-input-container">
645     <input type="text" id="chatInput" placeholder="Type your question here..." onkeypress="handleKeyPress(event)">
646     <button onclick="sendUserMessage()">Send</button>
647   </div>
648 </div>
649
650 <!-- Add these styles in your existing CSS -->
651 <style>
652 .chat-input-container {
653   display: flex;
654   gap: 10px;
655   padding: 10px;
656   border-top: 1px solid #eee;
657 }
658
659 #chatInput {
660   flex: 1;
661 }
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF () HTML
```



The screenshot shows a VS Code editor window with a file explorer on the left and a code editor in the center. The code is for a chatbot interface and includes the following:

```
function togglechat() {
  var chatbox = document.getElementById("chatbotWindow");
  chatbox.style.display = (chatbox.style.display === "none" || chatbox.style.display === "") ? "block" : "none";
}

async function sendUserMessage() {
  const input = document.getElementById('chatInput');
  const message = input.value.trim();

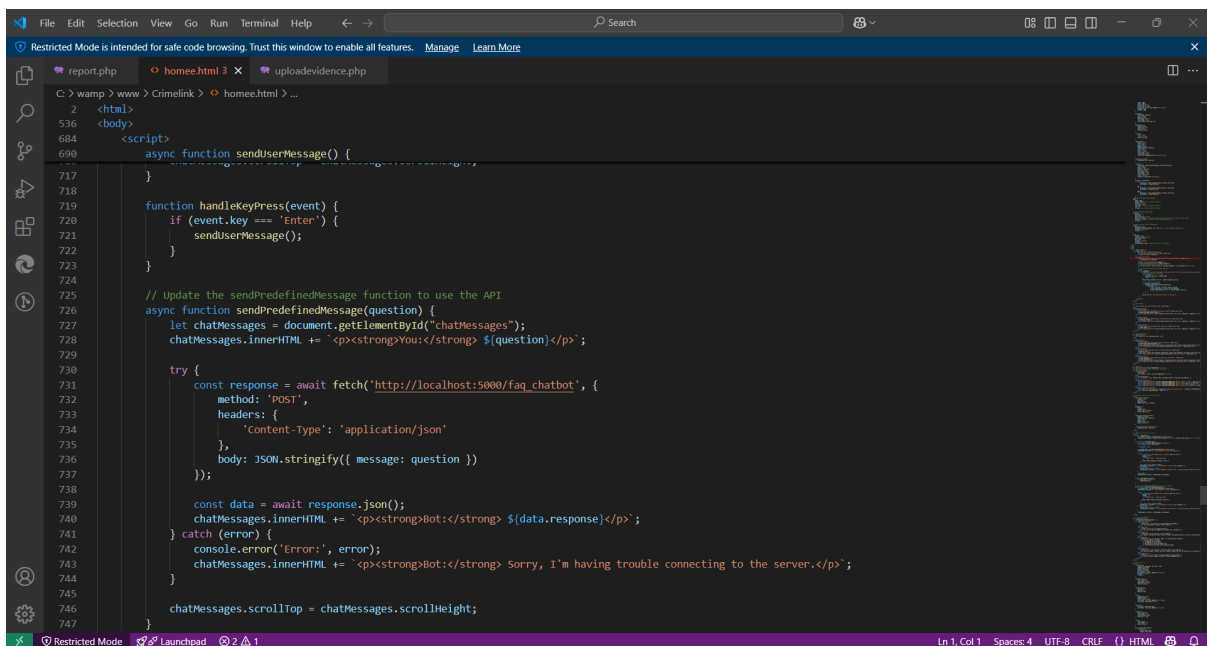
  if (!message) return;

  let chatMessages = document.getElementById("chatMessages");
  chatMessages.innerHTML += `<p><strong>You:</strong> ${message}</p>`;

  try {
    const response = await fetch('http://localhost:5000/faq_chatbot', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ message: message })
    });

    const data = await response.json();
    chatMessages.innerHTML += `<p><strong>Bot:</strong> ${data.response}</p>`;
  } catch (error) {
    console.error('Error:', error);
    chatMessages.innerHTML += `<p><strong>Bot:</strong> Sorry, I'm having trouble connecting to the server.</p>`;
  }

  input.value = '';
  chatMessages.scrollTop = chatMessages.scrollHeight;
}
```



The screenshot shows a VS Code editor window with a file explorer on the left and a code editor in the center. The code is for a chatbot interface and includes the following:

```
async function sendUserMessage() {
  // ... (code from previous screenshot) ...
}

function handleKeyPress(event) {
  if (event.key === 'Enter') {
    sendUserMessage();
  }
}

// Update the sendPredefinedMessage function to use the API
async function sendPredefinedMessage(question) {
  let chatMessages = document.getElementById("chatMessages");
  chatMessages.innerHTML += `<p><strong>You:</strong> ${question}</p>`;

  try {
    const response = await fetch('http://localhost:5000/faq_chatbot', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ message: question })
    });

    const data = await response.json();
    chatMessages.innerHTML += `<p><strong>Bot:</strong> ${data.response}</p>`;
  } catch (error) {
    console.error('Error:', error);
    chatMessages.innerHTML += `<p><strong>Bot:</strong> Sorry, I'm having trouble connecting to the server.</p>`;
  }

  chatMessages.scrollTop = chatMessages.scrollHeight;
}
```

4.3 Code for Police Dashboard main body

```
File Edit Selection View Go Run Terminal Help Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

home.html 2 policedashboard.html 5 x showevidence.php x
C:\wamp\www> Crimelink > policedashboard.html > ...

132 </body>
143 <!-- Main Content -->
144 <div class="container">
145   <div class="squarea">
146     
147     <h2>See Complaints</h2>
148     <button class="square-btn" onclick="window.location.href='see_complaints.php';">See Complaint</button>
149   </div>
150   <div class="squareb">
151     
152     <h2>Update Complaint Status</h2>
153     <button class="square-btn" onclick="window.location.href='view_status.php';">Update Status</button>
154   </div>
155   <div class="squarec">
156     
157     <h2>View Uploaded Evidence</h2>
158     <button class="square-btn" onclick="window.location.href='showevidence.php';">View Evidence</button>
159   </div>
160 </div>
161 </div>
162 </body>
163 </html>
164 <!-- Add this div after the header section -->
165 <div id="sosAlertBox" style="display: none; position: fixed; top: 100px; right: 20px; background: linear-gradient(135deg, #ff0000, #ff4d4d); color: white; padding: 10px; border-radius: 5px; font-weight: bold; font-size: 1.2em; text-align: center; width: 200px; margin: auto;">
166   <h3>SOS ALERT!</h3>
167   <p id="sosDetails">Emergency assistance needed!</p>
168   <button onclick="acknowledgesos()" style="background: white; color: red; border: none; padding: 10px 20px; border-radius: 5px; cursor: pointer; font-weight: bold;">Acknowledge</button>
169 </div>
170 <!-- Add this script before the closing body tag -->
171 <script>
172   function checkSOS() {
173     const sosData = localStorage.getItem('sosAlert');
174     if (sosData) {
175       document.getElementById('sosAlertBox').style.display = 'block';
176       document.getElementById('sosDetails').innerHTML = sosData;
177     }
178   }
179   checkSOS();
180 </script>
181 </body>
182 </html>
```

4.4 Code for showing evidence

```
File Edit Selection View Go Run Terminal Help Search
Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More

home.html 2 policedashboard.html 5 x showevidence.php x
C:\wamp\www> Crimelink > showevidence.php > ...

1 <?php
2 $servername = "localhost";
3 $username = "root";
4 $password = "";
5 $database = "crimelink";
6
7 // Create connection
8 $conn = mysqli_connect($servername, $username, $password, $database);
9
10 // Check connection
11 if (!$conn) {
12   die("Connection failed: " . mysqli_connect_error());
13 }
14
15 // Fetch data from evidence table
16 $query = "SELECT * FROM evidence";
17 $result = mysqli_query($conn, $query);
18
19 <?php
20 <!DOCTYPE html>
21 <html lang="en">
22 <head>
23   <meta charset="UTF-8">
24   <meta name="viewport" content="width=device-width, initial-scale=1.0">
25   <title>Evidence Records</title>
26   <meta charset="utf-8">
27   <link rel="icon" type="image/x-icon" href="logo.png">
28   <link rel="stylesheet" href="style.css">
29 </head>
30 <body>
31   <h2>Evidence Records</h2>
32   <table border="1">
33     <tr>
34       <th>Photo</th>
35       <th>Document</th>
36       <th>Report ID</th>
37     </tr>
38   </table>
39 </body>
40 </html>
```

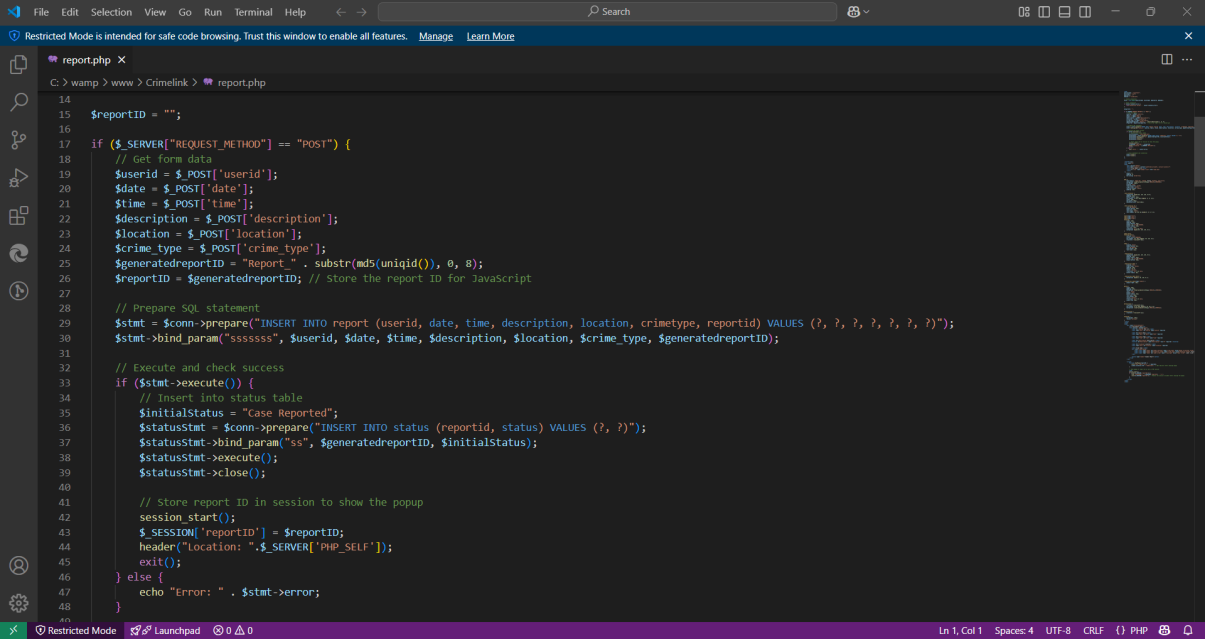
4.5 Code for evidence uploading

```
report.php uploadEvidence.php X
C:\> wamp > www > Crimelink > uploadEvidence.php
1  <?php
11 }
14 $reportid = $_POST['reportid'];
15
16 $doc_name = $_FILES['document']['name'];
17 $tmp = explode(".", $doc_name);
18 $document = round(microtime(true)).'.end($tmp);
19 $uploadpath = "uploads/".$document;
20 move_uploaded_file($_FILES['document']['tmp_name'], $uploadpath);
21
22 # $video_name = $_FILES['video']['name'];
23 # $video = $_FILES['video']['tmp_name'];
24 # $folder = "uploads/".$video_name;
25 # move_uploaded_file($video, $folder);
26
27
28 $image_name = $_FILES['image']['name'];
29 $tmp = explode(".", $image_name);
30 $image = round(microtime(true)).'.end($tmp);
31 $uploadpath = "uploads/".$image;
32 move_uploaded_file($_FILES['image']['tmp_name'], $uploadpath);
33
34 # $audio_name = $_FILES['audio']['name'];
35 # $tmp = explode(".", $audio_name);
36 # $audio = round(microtime(true)).'.end($tmp);
37 # $uploadpath = "uploads/".$audio;
38 # move_uploaded_file($_FILES['audio']['tmp_name'], $uploadpath);
39
40
41
42
43 $query = "insert into evidence (photo, document, reportid) values ('$image', '$document', '$reportid')";
44 if (mysqli_query($conn, $query))
45 {
46     // After successful upload
47     header("Location: submitEvidenceCode.html?success=true");
48 }
49 else {
50     // If failed to upload
51     header("Location: submitEvidenceCode.html?failure=true");
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

4.6 Code for Tracking citizen's complaint.

```
home.html.2 policedashboard.html.5 track.php X showEvidence.php
C:\> wamp > www > Crimelink > track.php
17 if ($SERVER["REQUEST_METHOD"] == "POST") {
31 if ($result->num_rows > 0) {
32 while ($row = $result->fetch_assoc()) {
47 foreach ($statusSequence as $status) {
51 } elseif (array_search($status, $statusSequence) < array_search($currentStatus, $statusSequence)) {
52 $stepClass = "completed";
53 }
54 }
55 // Timeline structure
56 $timelineHTML .= "<div class='track-step $stepClass'>";
57 $timelineHTML .= "<div class='step-icon'>";
58 switch ($status) {
59 case 'Case Reported': $timelineHTML .= "<img alt='Reported'>"; break;
60 case 'Pending': $timelineHTML .= "<img alt='Pending'>"; break;
61 case 'Complaint Verified': $timelineHTML .= "<img alt='Verified'>"; break;
62 case 'Evidence collected': $timelineHTML .= "<img alt='Collected'>"; break;
63 case 'Delegated to Further Authority': $timelineHTML .= "<img alt='Delegated'>"; break;
64 case 'In Progress': $timelineHTML .= "<img alt='In Progress'>"; break;
65 case 'Resolved': $timelineHTML .= "<img alt='Resolved'>"; break;
66 }
67 $timelineHTML .= "</div>";
68 $timelineHTML .= "<div class='step-content'><h4>$status</h4>";
69 if ($status == $currentStatus) {
70 $timelineHTML .= "<p><strong>Report ID:</strong> " . $row["reportid"] . "</p>";
71 $timelineHTML .= "<p><strong>Date:</strong> " . $row["date"] . " <strong>Time:</strong> " . $row["time"] . "</p>";
72 $timelineHTML .= "<p><strong>Crime Type:</strong> " . $row["crimetype"] . "</p>";
73 $timelineHTML .= "<p><strong>Location:</strong> " . $row["location"] . "</p>";
74 }
75 $timelineHTML .= "</div></div>";
76 }
77 $timelineHTML .= "</div>";
78 }
79 } else {
80 $timelineHTML .= "<p class='error-message'>No complaints found for this Report ID.</p>";
81 }
82 $stmt->close();
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }
```

4.7 Code for reporting cases



```
14  
15 $reportID = "";  
16  
17 if ($_SERVER["REQUEST_METHOD"] == "POST") {  
18     // Get form data  
19     $userid = $_POST['userid'];  
20     $date = $_POST['date'];  
21     $time = $_POST['time'];  
22     $description = $_POST['description'];  
23     $location = $_POST['location'];  
24     $crime_type = $_POST['crime_type'];  
25     $generatedreportID = "Report-" . substr(md5(uniqid()), 0, 8);  
26     $reportID = $generatedreportID; // Store the report ID for JavaScript  
27  
28     // Prepare SQL statement  
29     $stmt = $conn->prepare("INSERT INTO report (userid, date, time, description, location, crimetype, reportid) VALUES (?, ?, ?, ?, ?, ?, ?)");  
30     $stmt->bind_param("sssssss", $userid, $date, $time, $description, $location, $crime_type, $generatedreportID);  
31  
32     // Execute and check success  
33     if ($stmt->execute()) {  
34         // Insert into status table  
35         $initialStatus = "Case Reported";  
36         $statusStmt = $conn->prepare("INSERT INTO status (reportid, status) VALUES (?, ?)");  
37         $statusStmt->bind_param("ss", $generatedreportID, $initialStatus);  
38         $statusStmt->execute();  
39         $statusStmt->close();  
40  
41         // Store report ID in session to show the popup  
42         session_start();  
43         $_SESSION['reportID'] = $reportID;  
44         header("location: ".$$_SERVER['PHP_SELF']);  
45         exit();  
46     } else {  
47         echo "Error: " . $stmt->error;  
48     }  
49 }  
50
```

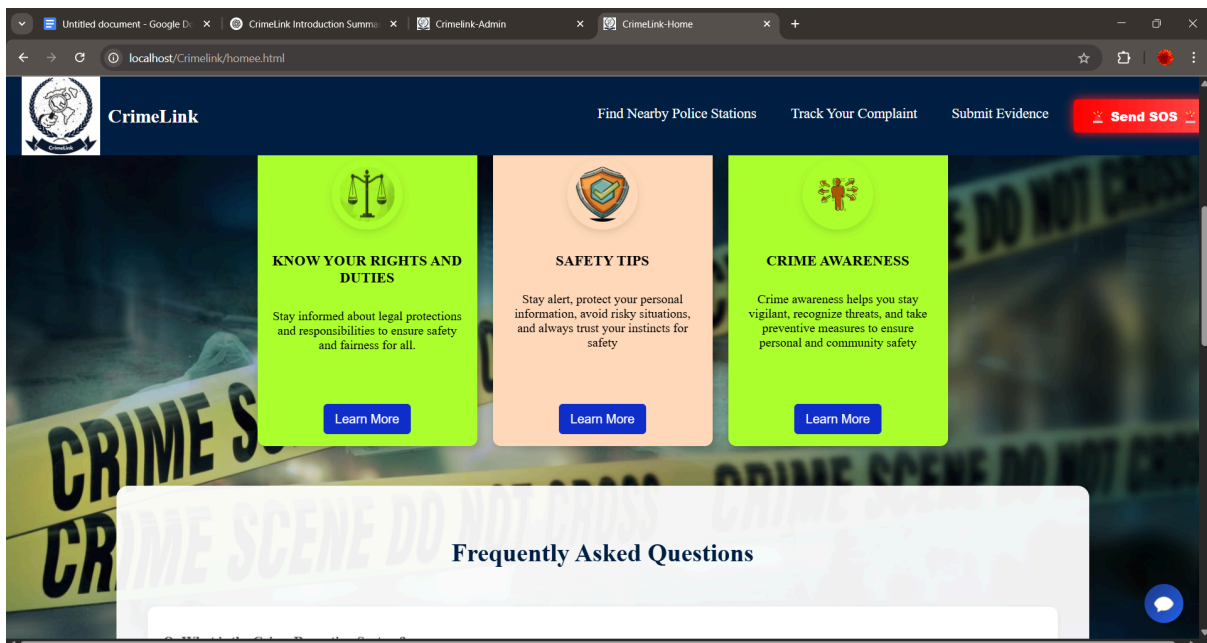
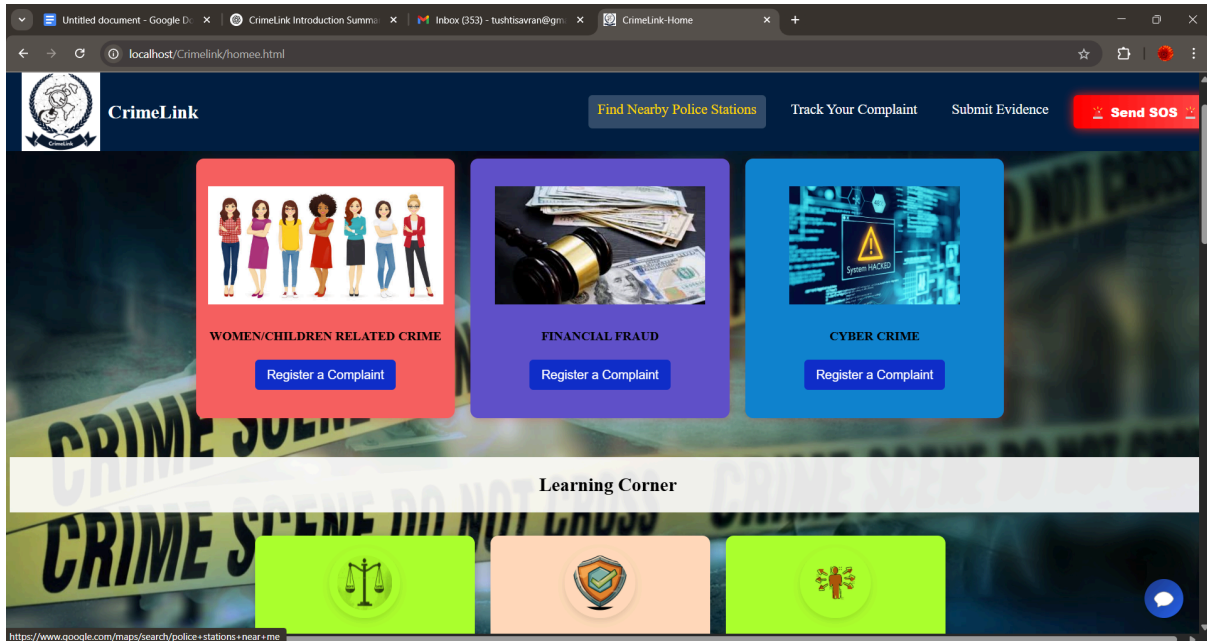
5. Testing

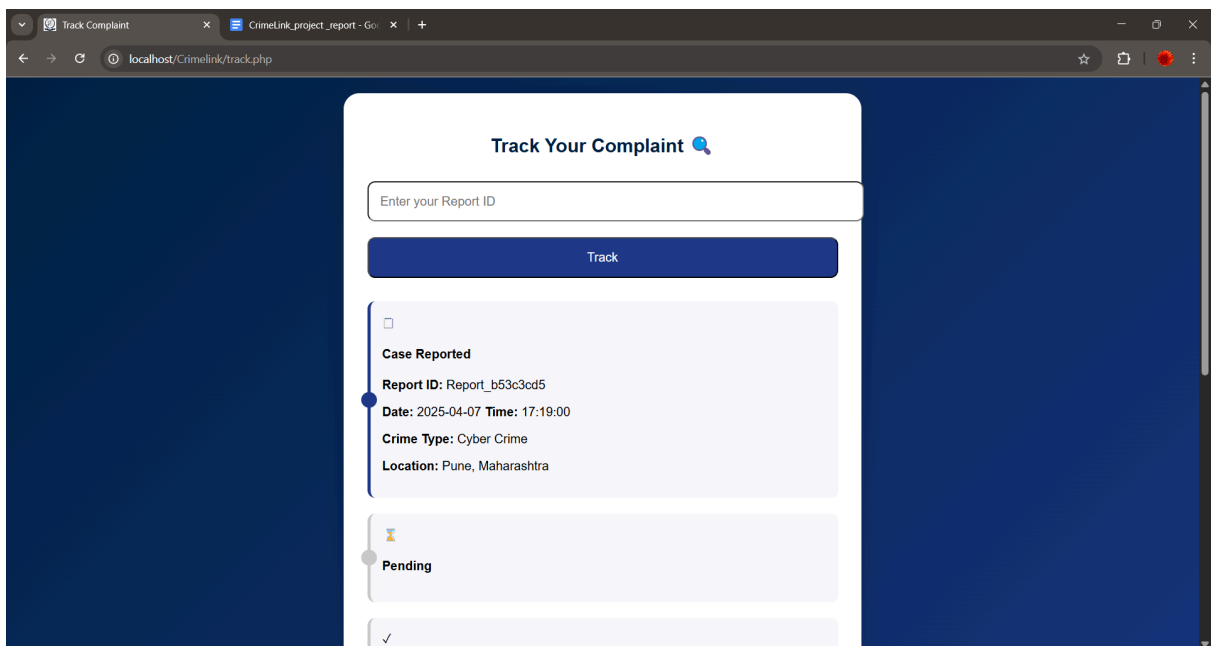
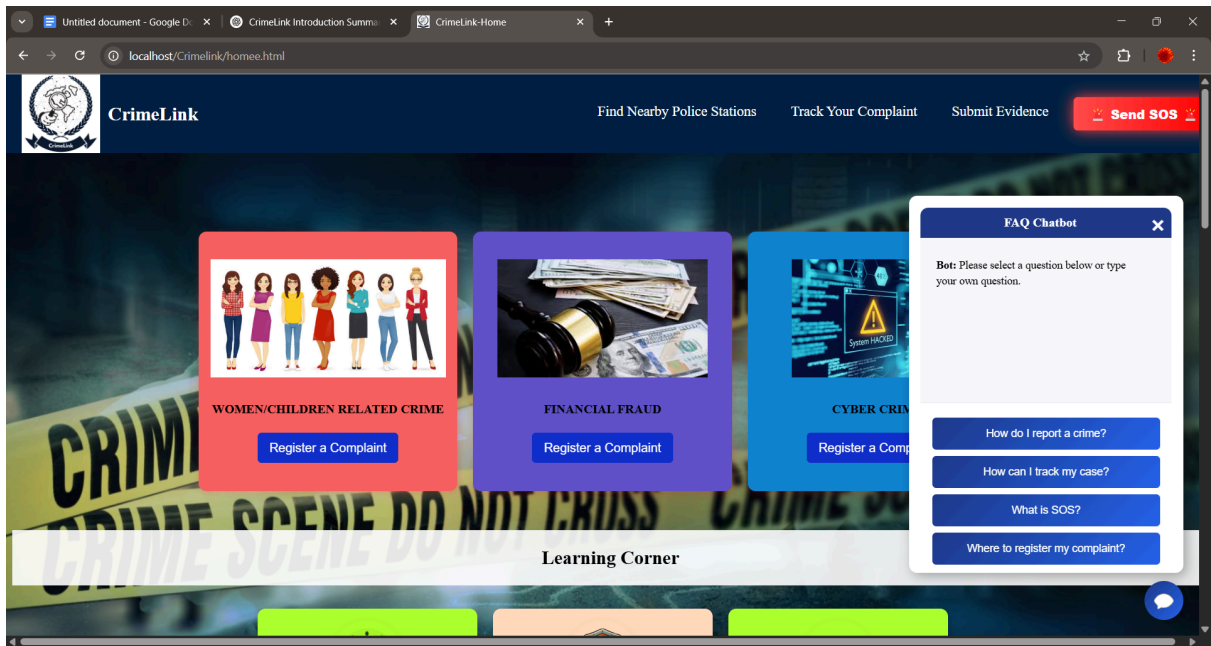
The following types of testing has been used in our project.

1. **Unit Testing:** Testing individual modules like login, registration, crime reporting forms.
2. **Integration Testing:** Testing interactions between modules like user login + report submission.
3. **User Acceptance Testing (UAT):** Ensuring the system is easy and satisfactory for users.

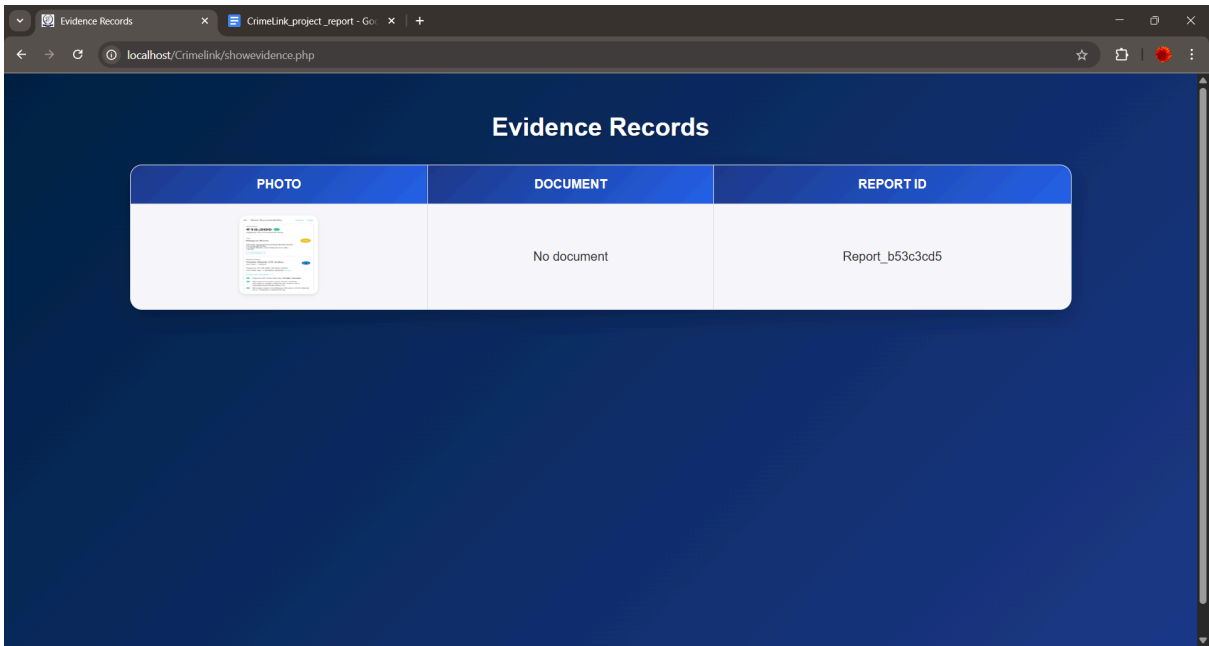
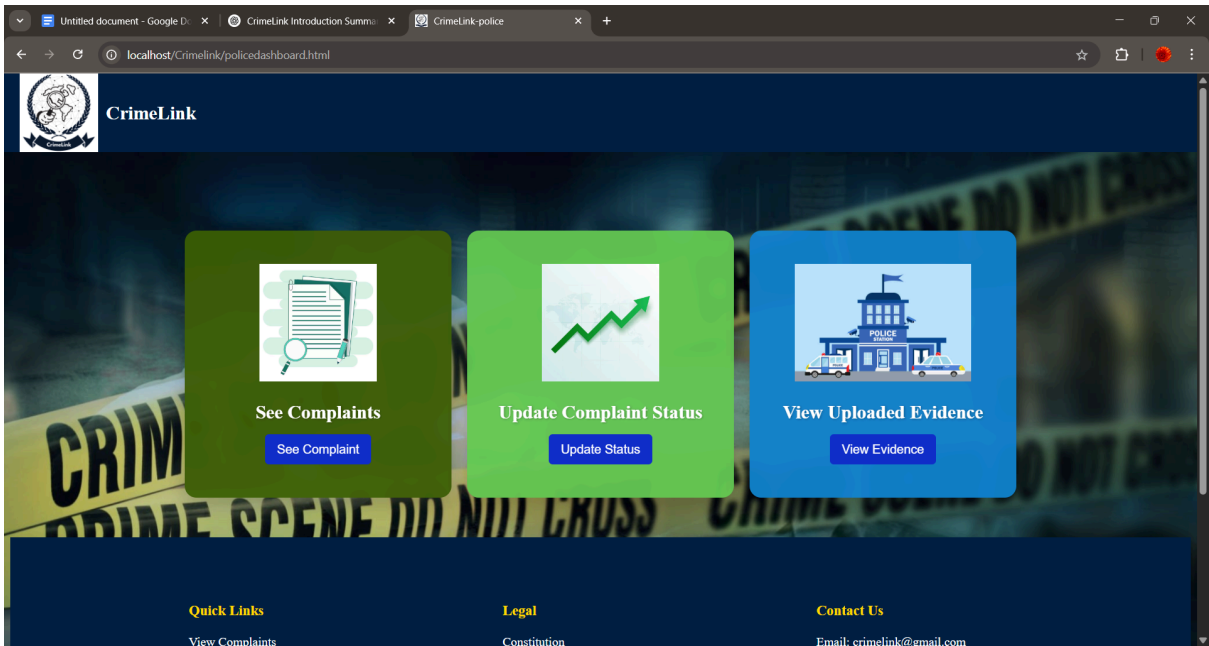
6. User Interfaces

6.1. Citizen interface

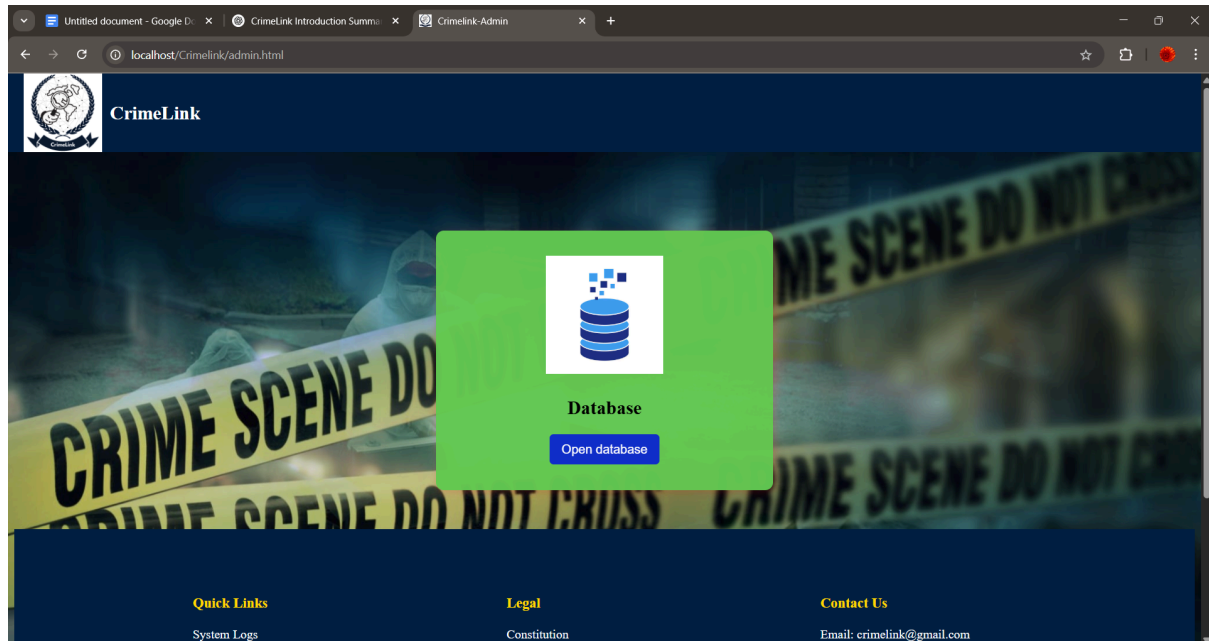




6.2.Police interface



6.3.Admin interface



7. Appendices

7.1 Screenshots of the Application

Screenshots have been included to provide a visual representation of the user interfaces and system features. These include:

- Citizen ,Police and Admin Dashboards
- Emergency SOS Feature Interface
- Chatbot Interaction Window

7.2 Database Schema Diagrams

Entity-Relationship (ER) diagrams and table representations are provided to show the structure of the underlying database. These diagrams illustrate:

- **Entities:** Users, Reports, Evidence, Status
- **Relationships:** One-to-many mappings between users and reports, reports and evidence

- **Primary and Foreign Keys** used for maintaining referential integrity

8. References

- PHP Official Documentation
- MySQL Official Documentation
- HTML5 and CSS3 W3Schools
- Online resources for chatbot integration (Python API)
- National Cyber Crime Portal -<https://cybercrime.gov.in/>